

Image Histogram and Histogram Equalization Report

M1496399 (阿洛)

Manual implementation in Python without calcHist or equalizeHist

Analyzed images

Image A - Night Sky with CCTV

Image B - Castle Scene

Image C - Feng Chia University Building

Course task	Histogram drawing and histogram equalization report
Methods required	Grayscale equalization, RGB channel equalization, and intensity equalization
Implementation note	All histograms and mappings were computed manually in Python
Attachments delivered	One .pdf report and one .py source file

1. Objective

This report fulfills the assignment requirement to read three images, draw their histograms, perform histogram equalization, and compare the results. The analysis uses the two given images and a chosen image by the student: a night-sky scene, a castle scene, and the Feng Chia University building.

2. Constraints and implementation strategy

The program does not use ready-made histogram or histogram equalization functions such as OpenCV `calcHist` or `equalizeHist`. Instead, it counts pixel values manually, computes the cumulative distribution function (CDF), builds the intensity mapping table, and applies the mapping to each pixel.

3. Processing tasks required by the assignment

For each test image, the report includes: (1) grayscale image histogram and grayscale equalization result, (2) RGB histogram equalization by processing R, G, and B channels separately and then merging them, (3) brightness/intensity equalization by equalizing the luminance map and merging it back into a color image, and (4) a short discussion comparing the two color enhancement approaches.

4. Core algorithm

The histogram of a grayscale channel is computed by counting how many pixels fall into each intensity level from 0 to 255. The equalization mapping is then derived from the cumulative distribution function. In notation:

```
hist[g] = number of pixels with intensity g
pdf[g]  = hist[g] / total_pixels
cdf[g]  = sum(pdf[0] ... pdf[g])
new[g]  = round((cdf[g] - cdf_min) / (1 - cdf_min) * 255)
```

For grayscale equalization, the above mapping is applied to the grayscale image. For RGB equalization, the same procedure is applied independently to the R, G, and B channels. For intensity equalization, the image is first converted to a luminance/intensity channel $Y = 0.299R + 0.587G + 0.114B$, Y is equalized, and the original RGB channels are rescaled by the ratio between the equalized and original intensity.

5. Program structure and explanation

The program is organized into small functions so that each processing stage is clear and traceable. The most important functions are shown below. The full source code is attached separately as a .py file and also included in Appendix A.

5.1 Manual histogram counting

```
def manual_histogram(channel):
    hist = np.zeros(256, dtype=np.int64)
    flat = channel.reshape(-1)
    for value in flat:
        hist[int(value)] += 1
    return hist
```

5.2 Manual equalization mapping

```
def cdf_mapping_from_hist(hist, total_pixels):
    pdf = hist.astype(np.float64) / float(total_pixels)
    cdf = np.cumsum(pdf)

    # Ignore zero-count leading bins to avoid mapping all dark pixels upward too aggressively.
    non_zero = np.nonzero(hist)[0]
    if len(non_zero) == 0:
        return np.arange(256, dtype=np.uint8), cdf
    cdf_min = cdf[non_zero[0]]
    denom = 1.0 - cdf_min
    mapping = np.zeros(256, dtype=np.uint8)
    if denom <= 1e-12:
        mapping = np.arange(256, dtype=np.uint8)
    else:
        for i in range(256):
            mapped = round((cdf[i] - cdf_min) / denom * 255.0)
            mapping[i] = np.uint8(min(255, max(0, mapped)))
    return mapping, cdf
```

5.3 RGB and intensity equalization

```
def manual_equalize(channel):
    hist = manual_histogram(channel)
    mapping, cdf = cdf_mapping_from_hist(hist, channel.size)
    eq = mapping[channel]
    return eq, hist, cdf, mapping
```

```
def equalize_rgb_channels(img):
    r_eq, r_hist, _, _ = manual_equalize(img[:, :, 0])
    g_eq, g_hist, _, _ = manual_equalize(img[:, :, 1])
    b_eq, b_hist, _, _ = manual_equalize(img[:, :, 2])
    merged = np.stack([r_eq, g_eq, b_eq], axis=2)
    return merged, (r_hist, g_hist, b_hist)
```

```
def intensity_equalize_by_luminance_scaling(img):
    # Intensity/brightness Y channel
    y = rgb_to_gray(img)
    y_eq, y_hist, _, _ = manual_equalize(y)

    img_f = img.astype(np.float64)
    y_f = y.astype(np.float64)
    y_eq_f = y_eq.astype(np.float64)

    ratio = np.ones_like(y_f)
    mask = y_f > 0
    ratio[mask] = y_eq_f[mask] / y_f[mask]

    out = img_f * ratio[:, :, None]
    out = np.clip(np.round(out), 0, 255).astype(np.uint8)
    return out, y, y_eq, y_hist
```

6. Quantitative summary

The table below uses grayscale standard deviation as a simple contrast indicator. A larger value generally means the intensity values are spread out more broadly.

Image	Original gray std.	After gray equalization	After intensity equalization
Image A - Night Sky with CCTV	26.40	73.46	72.15
Image B - Castle Scene	35.33	67.00	55.46
Image C - Feng Chia University Building	54.23	73.05	72.98

7.1 Image A - Night Sky with CCTV

The original image is strongly underexposed except for the lamp post and the brightest stars. Its grayscale histogram is concentrated in the low-intensity region, so equalization expands the dark range and reveals faint stars and cloud texture.

Grayscale histogram and equalization result

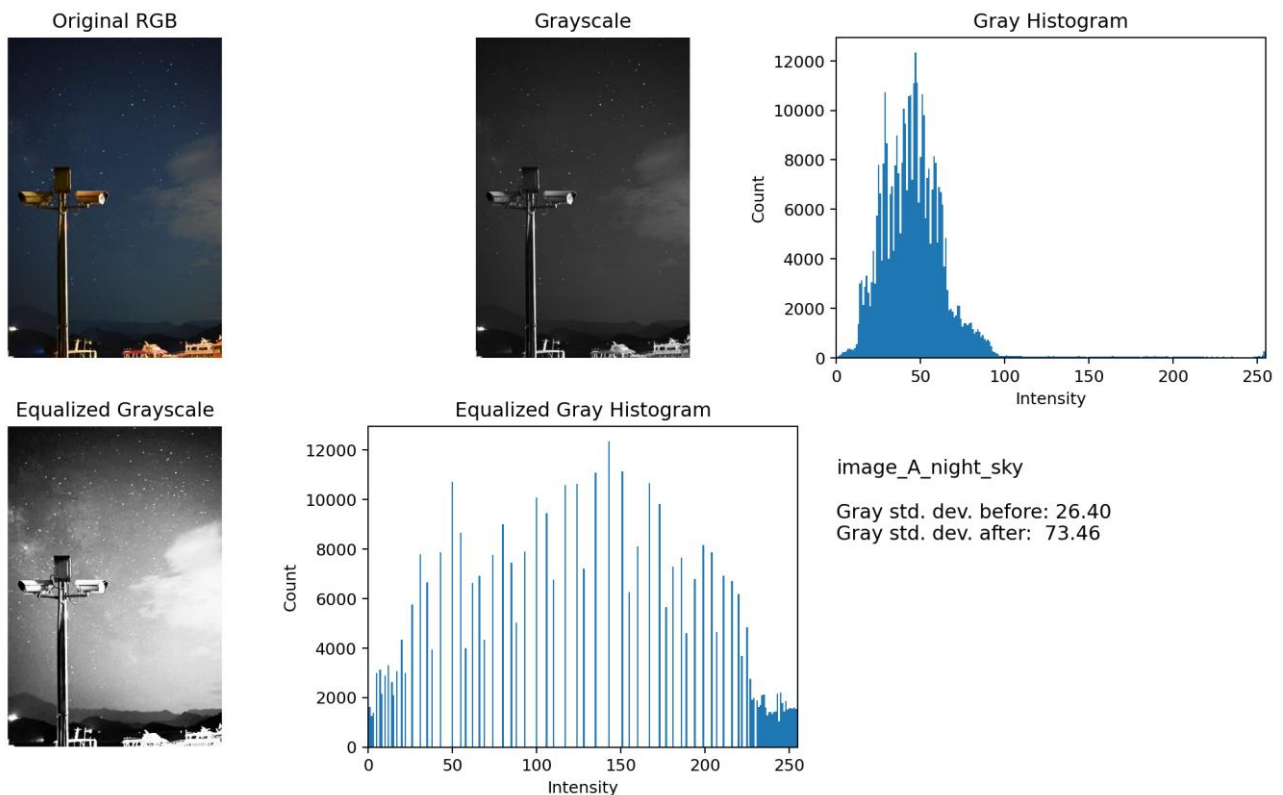


Image A - Night Sky with CCTV: original grayscale distribution and grayscale equalization result.

RGB channel-wise equalization result

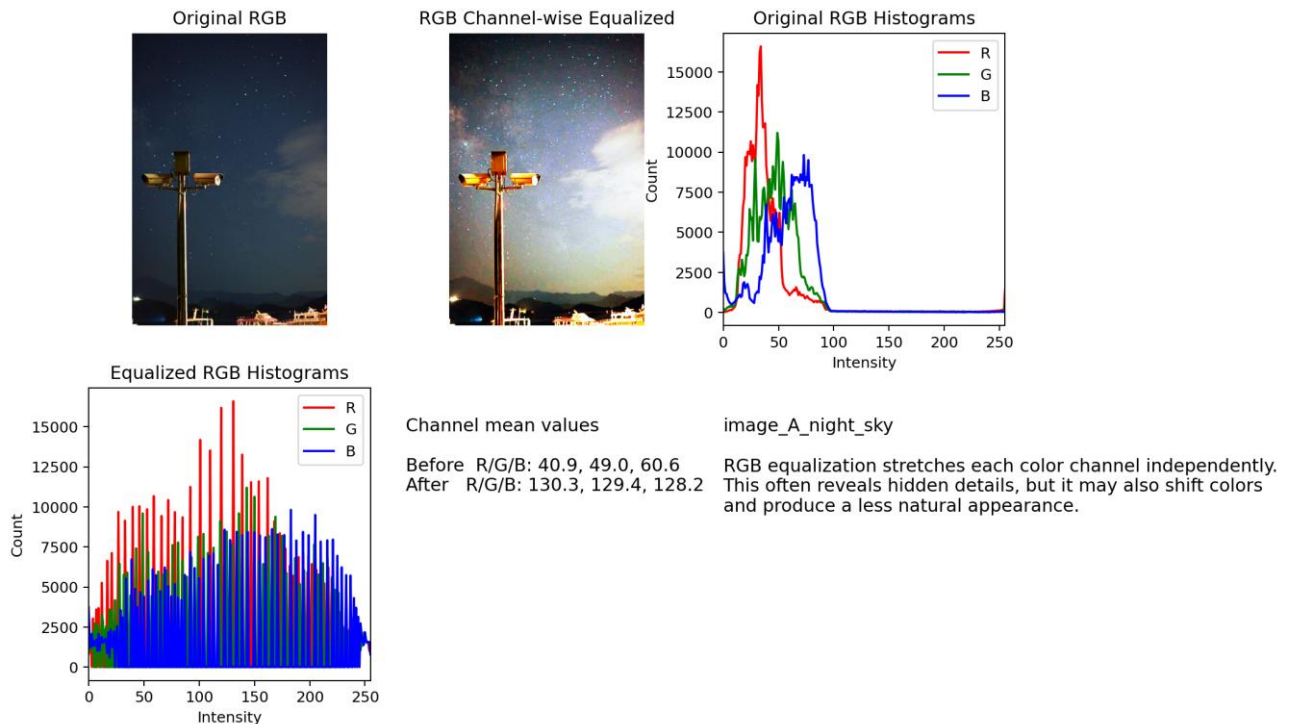


Image A - Night Sky with CCTV: separate R, G, and B equalization followed by channel merging.

Intensity equalization result

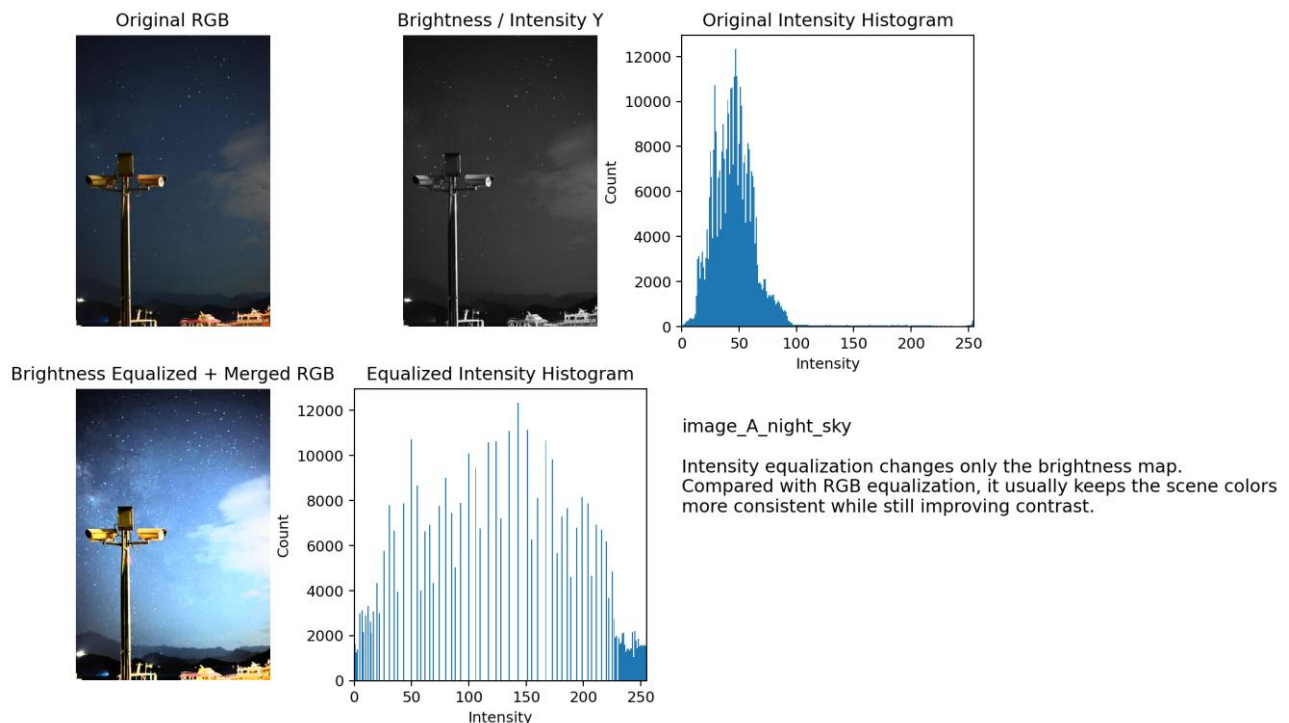


Image A - Night Sky with CCTV: luminance/intensity equalization and merged color result.

Discussion

Grayscale equalization produces the strongest global contrast gain. RGB equalization makes the sky much brighter but also shifts the color balance and introduces a less natural yellow-blue appearance. Intensity equalization keeps the scene closer to the original color mood while still improving visible detail.

7.2 Image B - Castle Scene

This image is dominated by blue light and large dark areas. The histogram is compressed near the dark end, which means many details are hidden before enhancement.

Grayscale histogram and equalization result

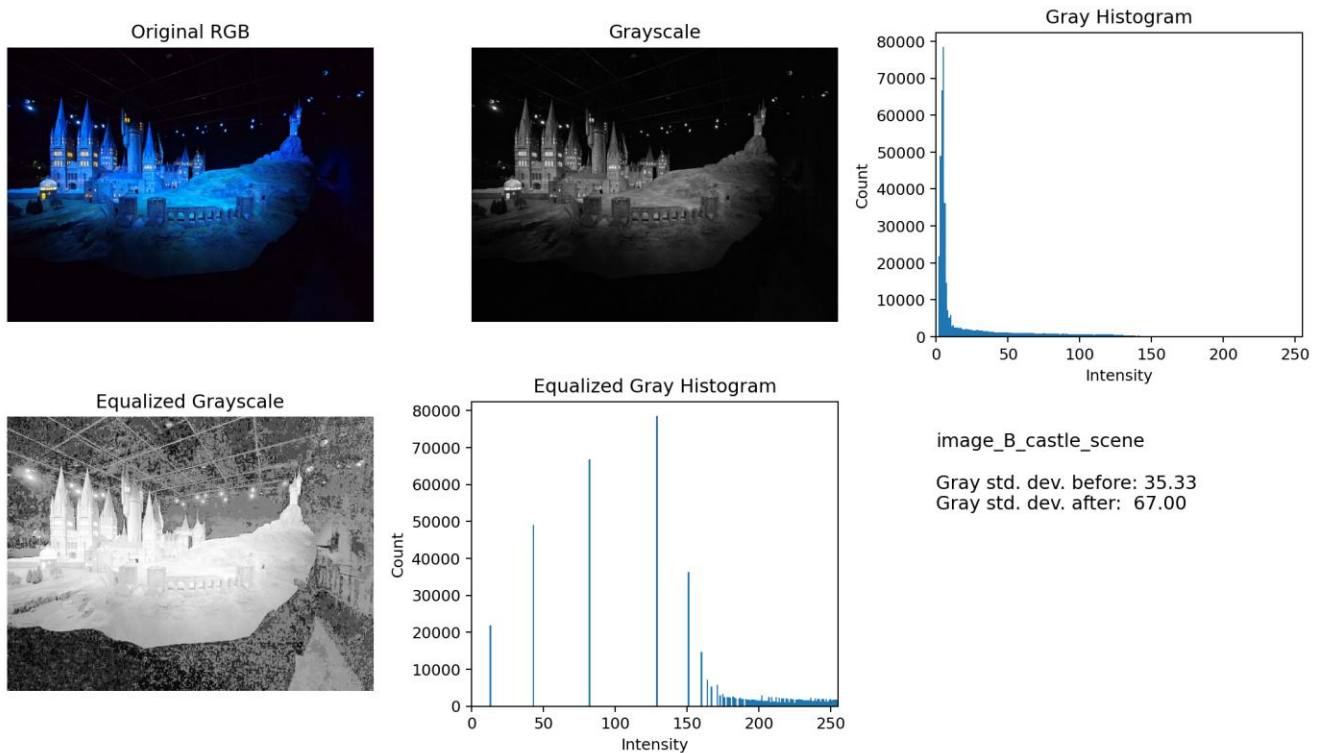


Image B - Castle Scene: original grayscale distribution and grayscale equalization result.

RGB channel-wise equalization result

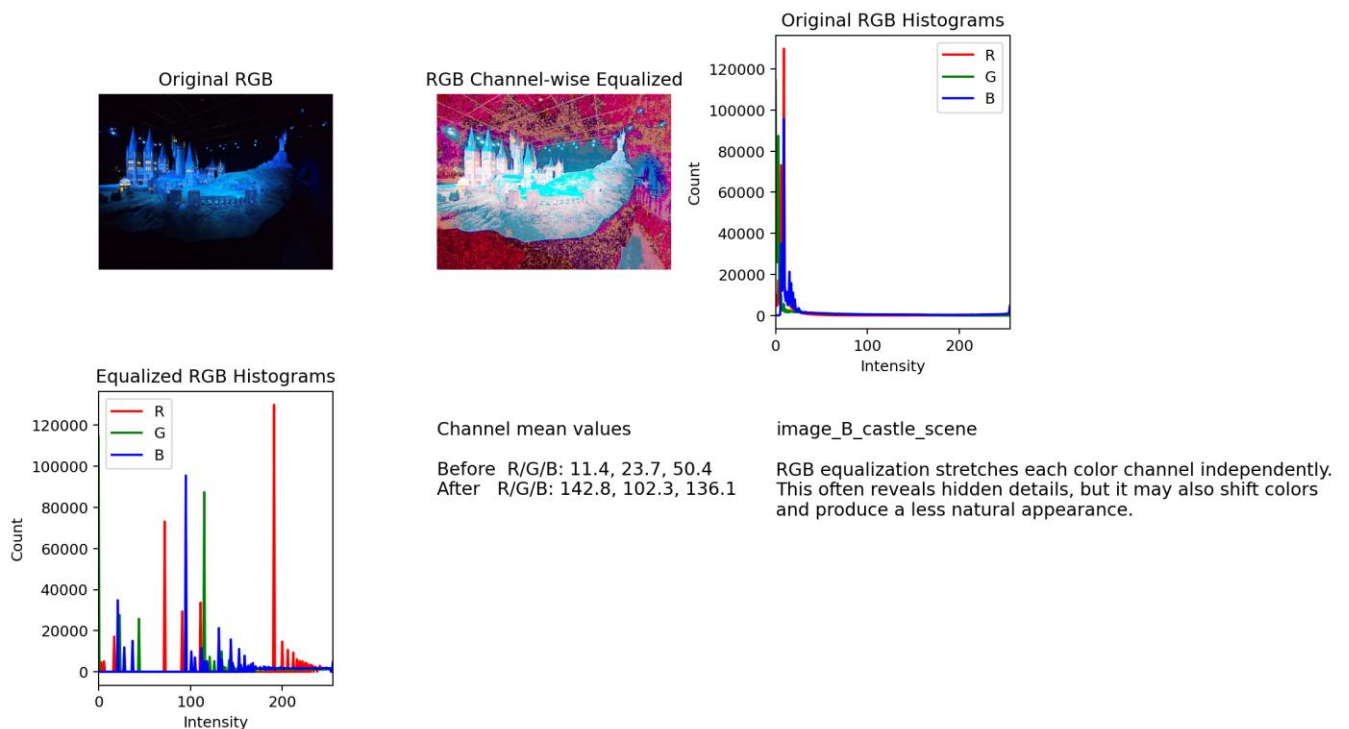


Image B - Castle Scene: separate R, G, and B equalization followed by channel merging.

Intensity equalization result

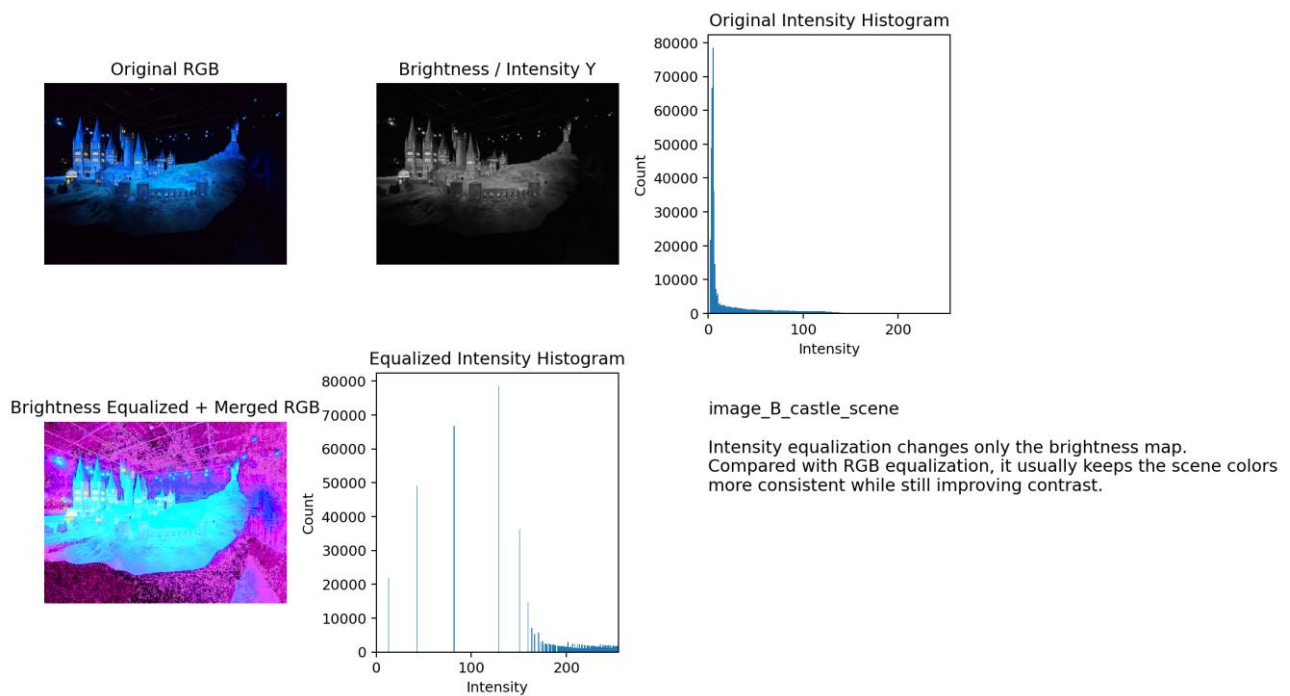


Image B - Castle Scene: luminance/intensity equalization and merged color result.

Discussion

Grayscale equalization uncovers building edges and terrain structure clearly. Channel-wise RGB equalization is the most aggressive, but it distorts the original blue atmosphere substantially. Intensity equalization gives a more balanced compromise: details improve, yet the scene remains more visually coherent than the RGB-only result.

7.3 Image C - Feng Chia University Building

The original image is already well exposed, with a broad histogram and visible separation between the bright building and blue sky.

Grayscale histogram and equalization result

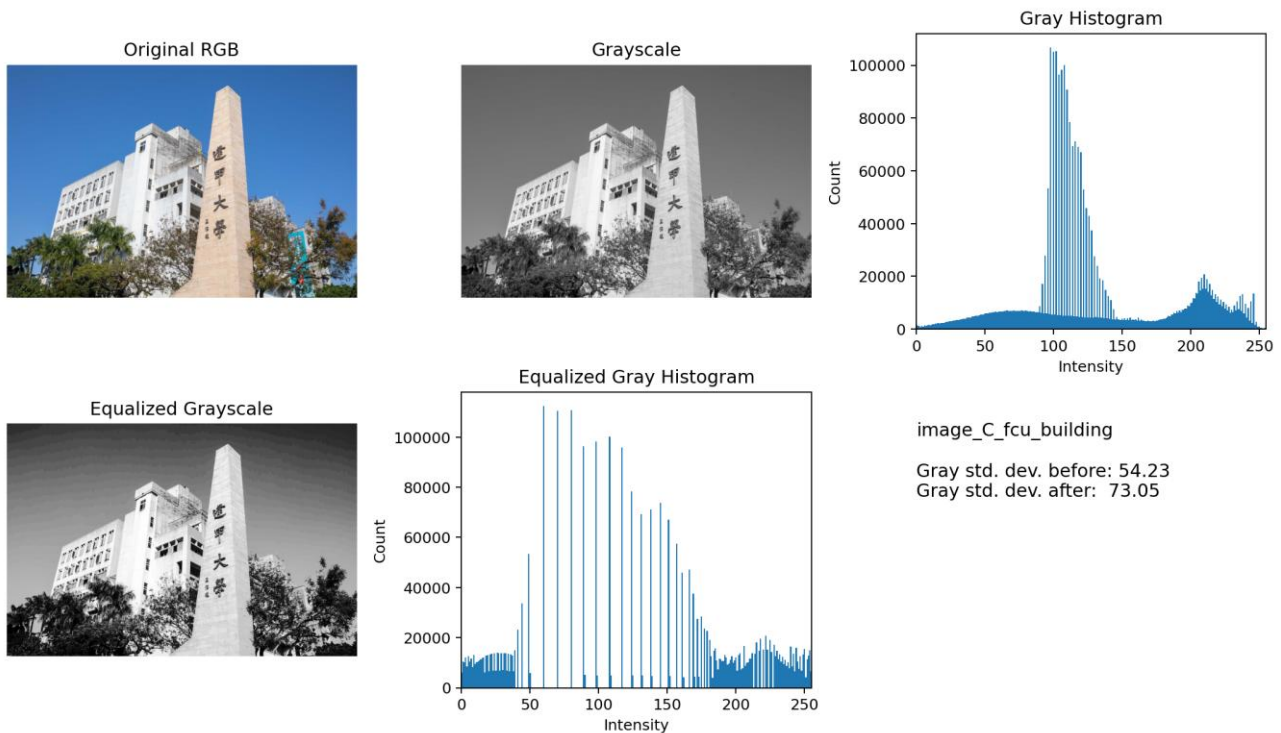


Image C - Feng Chia University Building: original grayscale distribution and grayscale equalization result.

RGB channel-wise equalization result

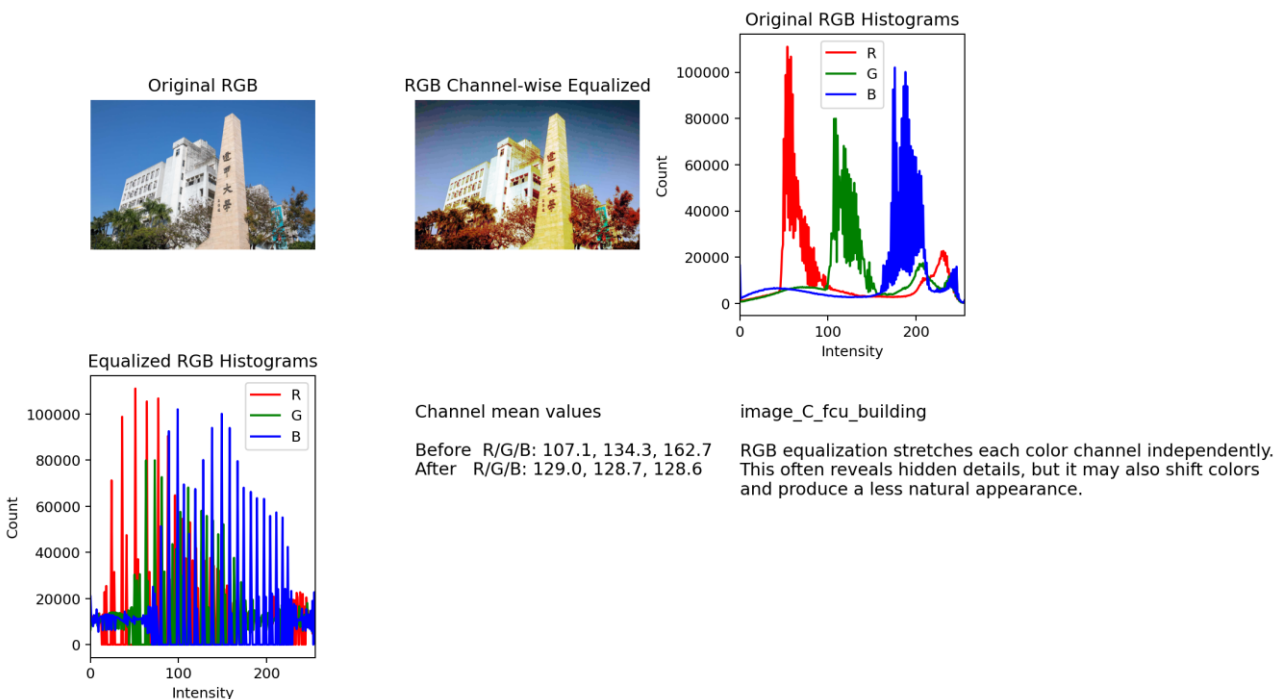


Image C - Feng Chia University Building: separate R, G, and B equalization followed by channel merging.

Intensity equalization result

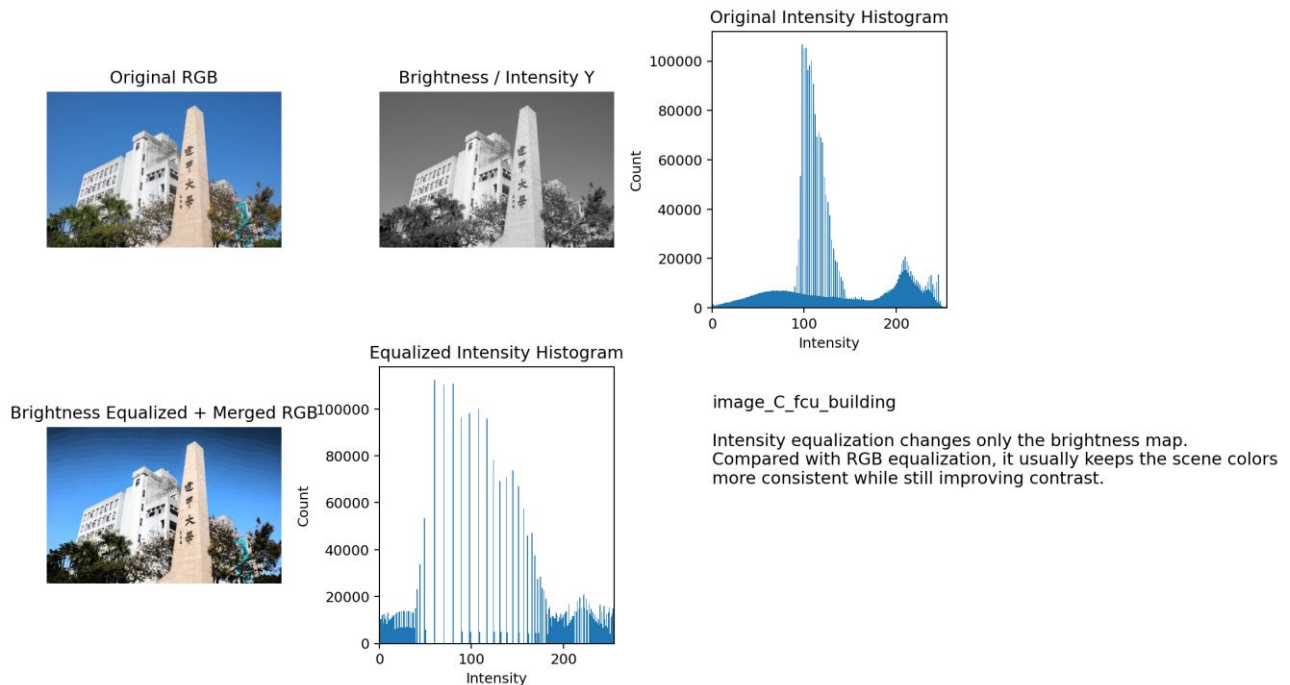


Image C - Feng Chia University Building: luminance/intensity equalization and merged color result.

Discussion

Because the input already has good contrast, equalization brings smaller practical benefit. Grayscale equalization makes the scene punchier, but it can also overemphasize shadows and highlights. RGB equalization increases saturation and may look unnatural. Intensity equalization produces only a moderate contrast change and preserves the most natural appearance among the enhanced color results.

8. Comparison of the two-color enhancement methods

The assignment specifically asks for a reflection on the difference between the two color methods. Based on the three test images, the difference can be summarized as follows:

Aspect	RGB channel equalization	Intensity equalization
Main operation	Equalizes R, G, and B separately	Equalizes only brightness/luminance
Contrast effect	Usually stronger and more aggressive	Usually moderate but stable
Color preservation	Can shift hue and saturation noticeably	Keeps colors closer to the original scene
Best use case	When maximum detail extraction is needed	When a natural-looking enhancement is preferred
Observed behavior in this report	Most dramatic for the night and castle scenes, but also the least natural	Better visual balance across all three images

9. Conclusion

Histogram equalization is highly effective for low-contrast and low-light images because it redistributes pixel values over a wider intensity range. In this report, the night-sky and castle images benefited the most from equalization because their original histograms were concentrated in the dark region. The campus building image, by contrast, was already well exposed, so equalization gave only limited practical improvement.

Among the two color-based methods, RGB channel equalization produced the strongest visual change but also the highest risk of unnatural color distortion. Intensity equalization preserved color relationships more successfully and therefore gave the most balanced result overall. For general enhancement tasks, intensity equalization is usually the safer choice, while RGB channel equalization is useful when stronger contrast enhancement is more important than color fidelity.

Appendix A. Full Python source code

```
from pathlib import Path
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

def load_rgb_image(path):
    """Load an image as an RGB uint8 numpy array."""
    return np.array(Image.open(path).convert('RGB'), dtype=np.uint8)

# -----
# Manual grayscale conversion
# -----
def rgb_to_grayscale(rgb):
    """Convert RGB image to grayscale manually."""
    gray = 0.299 * rgb[:, :, 0] + 0.587 * rgb[:, :, 1] + 0.114 * rgb[:, :, 2]
    return np.clip(np.round(gray), 0, 255).astype(np.uint8)

# -----
# Manual histogram
# -----
def compute_histogram(channel):
    """Compute 256-bin histogram manually without built-in histogram functions."""
    hist = np.zeros(256, dtype=np.int64)
    flat = channel.flatten()
    for pixel in flat:
        hist[int(pixel)] += 1
    return hist

# -----
# Manual histogram equalization
# -----
def equalize_channel(channel):
```

```

    """Equalize one uint8 channel manually using histogram + CDF."""
    hist = compute_histogram(channel)
    total_pixels = channel.size

    pdf = hist / total_pixels
    cdf = np.cumsum(pdf)

    # Normalize CDF to 0~255 mapping
    transform = np.round(cdf * 255).astype(np.uint8)
    equalized = transform[channel]
    return equalized, hist, compute_histogram(equalized)

# -----
# RGB channel-wise equalization
# -----
def equalize_rgb_separately(rgb):
    """Equalize R, G, B channels independently, then merge."""
    r_eq, _, _ = equalize_channel(rgb[:, :, 0])
    g_eq, _, _ = equalize_channel(rgb[:, :, 1])
    b_eq, _, _ = equalize_channel(rgb[:, :, 2])
    merged = np.stack([r_eq, g_eq, b_eq], axis=2)
    return merged

# -----
# Brightness-based equalization
# -----
def equalize_brightness(rgb):
    """Equalize grayscale intensity and rescale RGB while preserving color ratio."""
    rgb_float = rgb.astype(np.float32)
    gray = rgb_to_grayscale(rgb)
    gray_eq, _, _ = equalize_channel(gray)

    ratio = gray_eq.astype(np.float32) / np.maximum(gray.astype(np.float32), 1.0)
    result = rgb_float * ratio[:, :, None]
    return np.clip(np.round(result), 0, 255).astype(np.uint8)

# -----
# Save helpers
# -----
def save_image(array, path):
    Image.fromarray(array).save(path)

# -----
# Plot histogram helper
# -----
def save_histogram_plot(hist, title, output_path):
    plt.figure(figsize=(8, 4))

```

```

plt.bar(np.arange(256), hist, width=1.0)
plt.title(title)
plt.xlabel('Intensity Value')
plt.ylabel('Pixel Count')
plt.xlim(0, 255)
plt.tight_layout()
plt.savefig(output_path, dpi=150)
plt.close()

# -----
# Create montage for one image
# -----
def save_montage(original_rgb, gray, gray_eq, rgb_eq, brightness_eq, output_path, title):
    fig, axes = plt.subplots(2, 3, figsize=(14, 8))
    axes = axes.ravel()

    panels = [
        (original_rgb, 'Original RGB', False),
        (gray, 'Grayscale', True),
        (gray_eq, 'Grayscale Equalized', True),
        (rgb_eq, 'RGB Channel-wise Equalized', False),
        (brightness_eq, 'Brightness Equalized', False),
    ]

    for ax, (img, name, is_gray) in zip(axes, panels):
        if is_gray:
            ax.imshow(img, cmap='gray', vmin=0, vmax=255)
        else:
            ax.imshow(img)
        ax.set_title(name)
        ax.axis('off')

    axes[-1].axis('off')
    fig.suptitle(title, fontsize=14)
    plt.tight_layout()
    plt.savefig(output_path, dpi=150)
    plt.close(fig)

# -----
# Process one image
# -----
def process_image(name, image_path, output_dir):
    rgb = load_rgb_image(image_path)

    gray = rgb_to_grayscale(rgb)
    gray_eq, gray_hist_before, gray_hist_after = equalize_channel(gray)
    rgb_eq = equalize_rgb_separately(rgb)
    brightness_eq = equalize_brightness(rgb)

```

```

# Save images
save_image(rgb, output_dir / f'{name}_original.png')
save_image(gray, output_dir / f'{name}_gray.png')
save_image(gray_eq, output_dir / f'{name}_gray_equalized.png')
save_image(rgb_eq, output_dir / f'{name}_rgb_equalized.png')
save_image(brightness_eq, output_dir / f'{name}_brightness_equalized.png')

# Save histograms (grayscale only, per assignment)
save_histogram_plot(
    gray_hist_before,
    f'{name} - Grayscale Histogram Before Equalization',
    output_dir / f'{name}_gray_hist_before.png'
)
save_histogram_plot(
    gray_hist_after,
    f'{name} - Grayscale Histogram After Equalization',
    output_dir / f'{name}_gray_hist_after.png'
)

# Save montage
save_montage(
    rgb,
    gray,
    gray_eq,
    rgb_eq,
    brightness_eq,
    output_dir / f'{name}_montage.png',
    title=name.replace('_', ' ').title(),
)

# Simple contrast stats for discussion
summary = {
    'name': name,
    'gray_std_before': float(np.std(gray)),
    'gray_std_after': float(np.std(gray_eq)),
    'brightness_std_after': float(np.std(rgb_to_grayscale(brightness_eq))),
}
return summary

# -----
# Main program
# -----
def main():
    # Put your image files in the SAME folder as this .py file,
    # then rename them here if needed.
    images = {
        'image_A_night_sky': 'img1.png',
        'image_B_castle_scene': 'img2.png',
        'image_C_fcu_building': 'img3.png',
    }

```

```

script_dir = Path(__file__).resolve().parent
output_dir = script_dir / 'outputs'
output_dir.mkdir(parents=True, exist_ok=True)

summary_lines = []

for name, filename in images.items():
    image_path = script_dir / filename
    if not image_path.exists():
        print(f'[WARNING] File not found: {image_path}')
        print('Please place the image in the same folder as this script, or edit the
filename in main().')
        continue

    result = process_image(name, image_path, output_dir)
    summary_lines.append(
        f"{result['name']}: gray std before={result['gray_std_before']:.2f}, "
        f"gray std after={result['gray_std_after']:.2f}, "
        f"brightness-based gray std={result['brightness_std_after']:.2f}"
    )

summary_path = output_dir / 'summary.txt'
with open(summary_path, 'w', encoding='utf-8') as f:
    f.write('\n'.join(summary_lines))

print(f'Done. Outputs stored in: {output_dir}')
if summary_lines:
    print('Summary:')
    for line in summary_lines:
        print(line)

if __name__ == '__main__':
    main()

```